

A negative-control ledger for multi-stage numerical pipelines

— demonstrated on Ising/MERA

JianZhong Yan

(independent researcher)

mohedanovinita984@gmail.com

September 24, 2026

Abstract

We state an operational criterion for a genuine *negative control* in multi-stage numerical pipelines: **it must be able to fail when the conclusion does not hold**. Applying this criterion excludes positivity, finiteness, and non-emptiness checks, and sorts surviving controls into four types (repulsion, null-model, two-path, upper-bound). The ledger built from them carries a *completeness guard* that checks cited guards against the runtime registry, making the ledger hard to fake. Checks whose guarded proposition is measurably false are registered as *diagnostics* and excluded from the pass-rate denominator.

We demonstrate this method on a seven-layer Ising/MERA pipeline. **Results:** 54 of 54 scored assertions hold, 18 of 18 metrics meet target, and 12 negative controls cover 6 of 7 layers. The pass rate is a statement about guard execution, *not* about the truth of any physical emergence claim. The substrate is pinned by an external anchor—the ground-state energy density lies on the parameter-free CFT finite-size formula $e_\infty - \pi cv/(6L^2)$ at every L (relative residual 1.81×10^{-6} at $L=16$). A low-energy spectral ratio supplies a second ruler for the central charge (0.212% deviation), which is *independent of the entanglement-spectrum family* (though we note explicitly that the 0.212% figure and the “ratio $\rightarrow 8$ ” statement are the *same measurement*, not a double independent check). We report three *negative* results: one layer has no causal path to any downstream consumer; the bulk-geometry readout in the current fixed-wiring MERA implementation does not depend on the fitted state (it is an analytic property of the binary network layout); and two self-contradictory reporting conventions were corrected.

Boundary. “100%” means 54 scored assertions hold. It is *not* evidence that the system under study became better understood, and this paper claims no physical result beyond textbook Ising criticality. **Code availability:** <https://github.com/yanjianzhong/spiral-emergence>

1 Introduction

1.1 The problem: a sanity check is not a negative control

A layered pipeline derives quantities stage by stage. At each stage an assertion is made — “the spectrum is degenerate”, “the entropy is positive”, “the grid is fine enough” — and downstream stages consume the derived quantity. In practice these assertions are enforced by *positivity*, *finiteness*, and *non-emptiness* checks, often promoted from a hard **raise** into a readable boolean.

The difficulty is structural rather than clerical: such checks are *consistent with any conclusion*. “Schmidt rank ≥ 2 ” holds for a correct calculation and for a calculation that is wrong in every way that matters. A pipeline can therefore report a wall of green checks while every substantive claim in it is false, and nothing in the report distinguishes the two situations.

This paper is about a discipline that fixes that, and about what happens when the discipline is applied honestly — including where it *cannot* be applied.

1.2 Contributions

1. **A criterion** (Sec. 2.1) that separates genuine negative controls from true-by-construction checks, and that explicitly *excludes* the positivity/finiteness/non-emptiness family.
2. **A taxonomy** (Sec. 2.2) of four control types, each tied to a concrete implemented instance.
3. **A ledger with a completeness guard** (Sec. 2.3) that checks the ledger against the *runtime* registry of guards, rather than trusting the ledger’s own self-report. Crucially, this guard is itself falsifiable: if a cited guard does not exist or a layer is missing, it fails — making the ledger *hard to fake* (Sec. 7).
4. **A scoring/diagnostic split** (Sec. 2.4): checks whose guarded proposition is measurably false do not enter the denominator.
5. **A demonstration** (Secs. 3–4) on a seven-layer pipeline, reported together with its gaps, its negative results, and one real defect that the controls caught (Sec. 4.5).

1.3 Scope, and what this paper does not claim

Stated early, because it governs how every number below should be read:

- We do **not** claim that the seven-layer framework under study is established. Its independently audited maturity is $A \times 1$, $B \times 3$, $C \times 3$, $D \times 0$ (Sec. 4.1).
- We do **not** claim that any holographic bridge was established between layers 4 and 5 (Sec. 5.3); the tensor-network/holographic correspondence is considered only as background motivation following Swingle’s entanglement-renormalization approach to emergent spacetime [1].
- We do **not** claim that a 100% pass rate means the evidence became stronger (Sec. 5.1).
- The physics used as a substrate — critical Ising/CFT scaling — is **textbook material**, verified here as a substrate, not offered as a result.
- We do **not** claim that MERA cannot produce state-dependent geometry in general; we show only that in the present fixed-wiring binary implementation, the readout is an analytic property of the graph layout (Sec. 5.4).

The contribution on offer is methodological and engineering: the substrate is pinned down by external anchors, and the novelty, such as it is, lives in the *method layer*.

2 Method: criterion, taxonomy, and ledger

2.1 The criterion

Criterion. A check qualifies as a negative control only if there exists a concrete, reachable state of the pipeline in which *the conclusion it guards is false* and the check *fails*.

The criterion is applied as a filter, not as an aspiration. In the pipeline studied here, it removed an existing candidate — “the entanglement entropy is positive” at layer 4 — which had been counted as a control. Positivity holds for a wrong entropy as readily as for a right one; it is a sanity check and is now labelled as one. The same filter excludes the whole positivity / finiteness / non-emptiness family. This exclusion is recorded in the table header of the control registry in the source (`spiral_metric_v15._NEG_CTRL_TABLE`), so that the exclusion is a property of the code rather than of this prose.

2.2 Four types of control

Every control admitted by the criterion is assigned one of four types. Each type is defined together with the instance that actually implements it in the pipeline:

Type	Definition	Implemented instance
Repulsion	A quantity is pushed away from a value it would take if the conclusion were false; the observed displacement is the evidence.	Layer 4 bond dimension: when the bond dimension is forced below its lower bound, the downstream quantity degrades by $6.47\times$ (criterion $> 1.5\times$).
Null model	The measurement is compared against an ensemble generated to share the <i>nuisance</i> structure of the real object while destroying the structure under test.	Layer 5 bulk geometry: three null families $\times$ 3 instances — gnm (matched $ V $, $ E $; controls scale), ws (degree-matched small world; controls degree distribution), tree (balanced binary tree; controls the “nearly a tree” premise). Two-sided acceptance band [5%, 95%].
Two-path	The same quantity is obtained by a second, structurally different route; agreement is the evidence.	Five instances, spread across four layers (L2 $\times$ 2, L3, L5, L6) — for example the L2 spectral-gap degeneracy guard.
Upper bound	A computed quantity must not exceed an independently known bound.	Layer 7: a single elementary bound. This is the thinnest control in the ledger, and is reported as such.

Table 1: The four control types, with the instance that implements each. Type counts in the demonstrated ledger: repulsion 5, two-path 5, null-model 1, upper-bound 1 (12 total).

2.3 The ledger and its completeness guard

The ledger groups controls by physical layer, and each entry cites the guard name(s) that implement it. A ledger is a self-report, and a self-report can be written as a brochure: it can cite guards that do not exist, or quietly omit a layer.

The completeness guard closes that hole by checking the ledger against the *runtime registry* of guards rather than against its own text:

- every guard name cited by the ledger must exist in the live registry;
- each of the seven layers must appear exactly once.

In the reference run this guard reports **phantom references: none; missing layers: none**. Section 4.5 describes the defect this guard caught.

Note on what this guard is not. It is not a physical conclusion. It guarantees only that the ledger contains no phantom citation and no silently dropped layer. Its object is the honesty of the accounting, not the truth of the physics.

Why this guard is the method’s sharpest edge. The completeness guard is not merely a bookkeeping aid; it is the mechanism that makes the *method itself* falsifiable. A ledger that cites phantom guards or silently omits layers is, in effect, a falsified document — and this guard is designed to detect exactly that state. This is what distinguishes the present approach from every other pipeline-validation framework we are aware of: the validation framework is subject to the same logical standard it imposes on the pipeline. If the ledger’s own accounting is wrong, the guard fails *first*, before any physical claim is evaluated. The defect caught in Sec. 4.5 — where the completeness criterion itself was inverted — is the proof that this self-falsifiability is not a slogan: it caught a bug *in the validation layer*, which is exactly where bugs are hardest to see.

2.4 Scoring items versus diagnostics

Some guards protect propositions that are *measurably false* in the system as built, or that depend on a parameter rather than on the system. Reporting these as failures would be misleading, and silently dropping them would be worse. They are therefore registered as *diagnostics* and excluded from the pass-rate denominator.

In the reference run: 74 guards total, of which 54 are scored and 20 are diagnostics. **Of the 20 diagnostics, 17 fail by design.** The `expect_pass` flag is set explicitly at registration, so the distinction is a property of the code and survives any re-reading of the output. The rationale for each of the four most recent additions is recorded individually (Sec. 5.1).

Why this matters. A pipeline that counts “checks that hold” as evidence will inflate its score by admitting checks that cannot fail. Separating the denominator is the cheapest available defence, and it costs one boolean per guard.

3 The demonstration pipeline

3.1 Seven layers

The pipeline derives, layer by layer, from an exact-diagonalisation (ED) solution of the critical transverse-field Ising chain through entanglement spectra, bond dimensions, tensor-network bulk geometry, and a downstream scaling analysis. Each layer exposes (i) the proposition it asserts, (ii) the derived quantity it hands downstream, and (iii) its downstream consumers. The audit table records, per layer, which downstream consumer actually reads the derived quantity — this is what makes the gap in Sec. 5.2 visible.

3.2 Substrate and its constraint

The validation substrate is the critical transverse-field Ising chain: $c = 1/2$, $v = 2$, $e_\infty = -4/\pi$. Landau–de Gennes (Gray–Scott) pattern formation appears as a second, non-critical exhibit used for the layer-3 descriptor analysis.

One boundary deserves emphasis because it is easy to misattribute. The pipeline’s chain-length ceiling ($L \leq 16$ on the closed loop) comes from a **tensor-network library constraint**: the MERA construction used [2] requires L to be a power of two. It is *not* an exact-diagonalisation limit (sparse ED reaches $L = 20$ in 22.4 s) and *not* a laptop budget. This cause was misstated twice in the source comments before being corrected; it is reported here because the same misattribution is easy to make elsewhere.

4 Results

4.1 Coverage of the ledger, and its gap

The ledger’s 12 negative controls cover **6 of the 7 layers**; the gap is [’L1’]. The strongest layer is L5 (three null families $\times$ 3 instances, two-sided band); the thinnest is L7, with a single upper-bound control. The per-layer coverage guard is *deliberately not scored*: coverage is a to-do list, not a property of the model, and scoring it would be grading the checklist.

4.2 External anchor: the $1/L^2$ coefficient, at every L

The substrate is pinned by a comparison to a formula with *no free parameters* [3]:

$$\frac{E_0(L)}{L} = e_\infty - \frac{\pi cv}{6L^2}, \quad c = \frac{1}{2}, \quad v = 2, \quad e_\infty = -\frac{4}{\pi}. \quad (1)$$

E_0 comes from exact diagonalisation; the constants on the right do not enter its computation.

Quantity	Reference run	Note
Scored assertions	54/54 = 100.00%	see Sec. 5.1
Diagnostics excluded	20 (17 failing by design)	<code>expect_pass=False</code>
Metrics meeting target	18/18	
Negative controls	12	repulsion 5 / two-path 5 / null 1 / bound 1
Layer coverage	6/7, gap ['L1']	Sec. 5.2
Ledger completeness	no phantom refs, no missing layer	Sec. 2.3
Wall-clock (single laptop)	1245 s	order-of-magnitude only; was 1138 s in an earlier run

Table 2: Reference-run summary. Values transcribed from the pipeline log of the reference run.

L	relative residual vs. Eq. (1)	residual-inverted power p
8	2.88×10^{-5}	4.007
$\vdots$	$\vdots$	$\vdots$
16	1.81×10^{-6}	4.004
18	1.13×10^{-6}	4.002

Table 3: The residual decreases monotonically with L , and the residual-inverted power is $p = 4.00$ throughout — the expected leading correction. The anchor script carries a `--selftest` that demonstrates the comparison is not identically true.

Honest boundary. This is stronger than the anchor already present in the code, which compared only the *extrapolated* e_∞ against $-4/\pi$ (deviation 5.0×10^{-6}); here the comparison is against the *per- L* $1/L^2$ coefficient, which involves the product cv . But it remains a statement that *exact diagonalisation was performed correctly*. It is textbook content, not a result of this paper.

4.3 A second family that is genuinely independent

A second estimator for the central charge uses the low-energy spectral ratio $(E_2 - E_0)/(E_1 - E_0)$, which is independent of v :

Quantity	Value	Note
ratio at $L=8$	7.9231	monotone toward 8
ratio at $L=16$	7.9807	
c interval	[0.49894, 0.50034]	max deviation 0.212%
Rényi family, same pipeline	—	deviation 7.97%

Table 4: Low-energy spectral estimator versus the entanglement-spectrum family.

Two hard boundaries, both required reading. (i) The estimator is a self-consistency check, not an assumption-free measurement: the operator content is identified before use (E_1 is the R -sector vacuum, $\Delta_1 = 1/8$, at every point; E_2 is the NS two-particle state, $\Delta_2 = 1$), so $\Delta_1 = 1/8$ enters the construction. (ii) The readout does **not** converge monotonically in L — $L=16$ is the worst point — because the residual is an $O(1/L)$ finite-size correction. One may say “consistent to 0.21%”; one may **not** say “extrapolates to 0.5”.

A trap worth naming explicitly. The 0.212% figure and the “ratio $\rightarrow 8$ ” statement are *the same measurement*: 0.212% is the 0.24% by which the ratio misses 8. It must therefore **not** be written as “the spectral family and the Rényi family corroborate each other”. The correct statement is that the spectral family is *independent of the entanglement-spectrum family*. Those are different claims, and only the second is supported.

4.4 Qualitative distinction: area law versus logarithmic law

At the critical point the entanglement scaling is logarithmic; away from it, the area law holds. The measured peak of $|c_{\text{scaling}}|$ sits at $h/J = 1.00$; at the critical point $c_{\text{scaling}} = 0.4976$ ($R^2 = 1.0000$), while the off-critical points give 0.0007 ($h/J = 0.5$) and -0.0187 ($h/J = 1.5$). A shape test on $S(\ell)$ gives $\Delta\text{AIC} = 76.14/10.61/16.35$.

Honest boundary. The shape fit ($c = 0.5072$) and the L -scaling fit ($c = 0.4976$) use *the same* S data; this is self-consistency, not a cross-check. The lever arm $L : 8 \rightarrow 16$ spans only 1.00 octave in $\log_2 L$. This is a *qualitative* distinction, not a quantitative extrapolation.

4.5 Negative controls caught a real defect

The strongest argument that the controls are not decorative is that one of them caught a defect in which **a completeness criterion was inverted**.

At three places the ledger wrote “the XX negative control has been run” — a *self-reported* name — into the same tuple as the actually-executed guard names. Such names are registered only on an early-return branch, and therefore *never enter the registry when the control executes normally*. The consequence was exactly backwards: the completeness check **failed in the healthy case and passed when the control was missing**. Phantom references: 3 before the fix, none after.

What makes this worth reporting. The defect was found only by a *full pipeline run*. Before that, a source-text-versus-regex audit had reported 26/26 correct. A source-text audit is necessary but not sufficient: it cannot see runtime registration paths. The defect was introduced by the present work, not inherited; it is reported because the failure mode — a check that passes for the wrong reason — is the exact failure this entire method exists to prevent.

5 Honest boundaries and negative results

This section is a first-class section, not a discussion appendix. The method’s central claim is that a pipeline should account for what it *cannot* check; if that accounting is demoted to a footnote, the method has not been applied.

5.1 Twenty diagnostics, seventeen failing by design

Log line, quoted: guards 54/54 = 100.00% (plus 20 diagnostics excluded, of which 17 did not pass).

“Did not pass” is not a synonym for “defective”. These 17 are *archived negative results*. The four most recent additions, each with its reason for exclusion from the denominator:

- **A third corroborating path does not exist.**
- **The MERA bulk readout does not depend on the state** (Sec. 5.4).
- **That layer has no causal path** (Sec. 5.2).
- **The spectral radius is parameter-dependent**, not a property of the system.

By contrast, the one addition that *does* enter the denominator can fail: if the bond-dimension derivation were replaced by a hand-picked constant, its control would flip. The threshold there ($> 1.5\times$) is an order-of-magnitude comparison, and the line it references is not a strict lower bound — both facts are recorded in the guard’s own note.

5.2 Gap 1 — a layer where no control can be constructed

For layer L1 the ledger reports no negative control. The cause has been traced, and it is not “nobody got round to it”.

The quantity L1 hands downstream, `equal_state`, is **never consumed anywhere in the repository**: it appears exactly once, in its own `return` statement. Sibling quantities are no better — a condition number is reported but nothing is checked against it; a dimension is fed into a downstream derivation whose result is *printed* but never compared against $2^{L_{\text{chain}}}$.

There is no quantity to push away from, because there is no downstream consumer to be affected. The gap is registered as a *source-level attribute-access scan*, which is falsifiable in the useful direction: should anyone connect the quantity, the scan flips. A smoke test injects a fake consumer to demonstrate that the scan is not identically false.

This gap is “empty for a reason”, not “left unfilled”. It has deliberately *not* been entered into the L1 row of the ledger — doing so would pretend that L1 has a negative control.

5.3 Gap 2 — a bridge that is wired, and still carries nothing

An earlier internal note claimed that the L4→L5 bridge was *empty*. Direct reading of the call sites shows that claim was wrong: the bridge is wired end to end — the MERA graph is built, the curvature report is computed from it, the forman value is derived from it, and the control family is constructed on it.

The controls are not missing either; they are the strongest in the ledger (three null families $\times$ 3 instances, two-sided band). Measured outcomes:

- **F6b** (the MERA bulk geometry lies within the control distribution): *passes*. Forman percentile 33%, Ollivier percentile 33% inside the [5%, 95%] band.
- **F6c** (the MERA geometry is more negatively curved than *all* controls): *false, by design excluded*. Per family, Forman: `gnm` -3.553 , `tree` -0.867 , `ws` -0.194 , versus MERA -2.242 . The `gnm` control is *more* negative than the MERA graph — the escalation claim does not even have the right sign.
- **F6d** (the readout depends on the state, or on the bond dimension): *false, by design excluded*. See Sec. 5.4.

So the gap is neither “not wired” nor “no control available”. It is a third kind: **no upstream-derived quantity flows along the wire**. The quantity L4 derives ($\chi_{\text{dev}} = 4$, from the area law) is consumed elsewhere — its consumers were enumerated by search — and *none of them leads to the L5 construction*, which hard-codes its own bond dimension.

5.4 The “geometry” readout is an analytic property of the wiring

Layer 5’s bulk-geometry readout is computed on a MERA tensor-network graph. Within the present implementation—a fixed-wiring binary MERA where the graph builder reads only the index map—the readout does not depend on the fitted state, seed, or bond dimension (Table 5). Three measurements show this:

Varied	Method	Result
seed (0, 1, 7)	rebuild and compare edge sets	identical
bond dimension χ (2, 4, 8)	rebuild and compare edge sets	identical
state	the graph builder reads only the index map; the fit routines only replace tensor data	wiring

Table 5: The MERA bulk graph does not depend on the state, on the seed, or on the bond dimension in the current fixed-wiring implementation.

Honest boundary. In homogeneous MERA tensor networks, many structural properties are predetermined by the network geometry and are largely independent of variational parameters [4]. This is a normal property of the architecture; what it means here is that the readout is a property of the **binary-layout wiring**, and the upstream claim must be re-worded accordingly. It is therefore *not* a state-dependent emergent geometry under the present implementation. Whether a modified architecture (e.g., cMERA, state-dependent disentanglers, or gauged/SymTFT lift) restores state dependence is left to future work following Swingle’s entanglement-renormalization approach [1]. Reported values: $|V| = 62$, $|E| = 91$, $\bar{d} = 2.935$, **triangles** = 0, Forman mean -2.242 , Ollivier mean -0.3297 , percentiles 33%/33%.

Distinguishing the two gaps. They must not be merged into one “gap” in a report:

	L1	L4→L5
path	no causal path (unconsumed quantity)	path complete end to end
control	cannot be constructed (nothing to push)	constructed; strongest in the ledger
what is missing	the <i>tool</i>	the <i>alignment</i>

Table 6: Two structurally different gaps.

In one sentence: **L1 is empty for a reason; L4→L5 is filled generously, but on a different question than the one the framework asks.**

5.5 Two rulers for a length range

When reporting a range of chain lengths, the ratio and the logarithmic span must be reported separately. $L : 8 \rightarrow 16$ is a factor of 2.00 but spans only **1.00 octave** in $\log_2 L$; $L : 8 \rightarrow 18$ is $2.25\times$ but 1.17 octaves. Reporting the first number while implying the second is an overstatement. The ceiling itself comes from the tensor-network library constraint discussed in Sec. 3.2.

5.6 Two self-contradictory conventions, corrected

Two reporting conventions internal to the project contradicted each other and were corrected rather than reconciled by wording: (i) a claim that a scatter statistic was “inter-seed spread on the reference side” — contradicted by the archived fields; (ii) a claim that two routes “corroborate each other” — contradicted by the fact that they are the same measurement (Sec. 4.3). Both are recorded as corrections, with the original documentation left unedited and the discrepancy noted.

6 Related work

Tensor networks and MERA. The Multi-scale Entanglement Renormalization Ansatz (MERA) was introduced by Vidal [5] and developed into a binary-graph implementation by Evenbly and Vidal [2]. Its application to holographic duality and emergent geometry was motivated by Swingle [1], who noted that entanglement renormalization naturally produces a hyperbolic geometry in the bulk. In homogeneous MERA, many structural properties are predetermined by the network geometry and largely independent of variational parameters [4], which is the context for our negative result in Sec. 5.4.

Negative controls in experiment. The criterion used here is the ordinary logic of a controlled experiment, transplanted into software. The transplant is the point: in a wet-lab experiment a negative control is a physical sample; in a pipeline it must be an *executable* alternative, and executable alternatives are easy to omit without noticing.

Property-based testing and metamorphic testing. The repulsion and two-path types are close relatives of metamorphic relations and property-based tests, in that they assert a relation that must hold across a transformation rather than a fixed expected output. The difference is

one of framing: these controls are attached to *scientific claims about a physical system*, and their failure is meant to propagate into how loudly the claim is stated.

Reproducibility infrastructure. Workflow engines and provenance systems record *what ran*. This work is about a complementary question: *what would have to be true for the result to be wrong*, and whether the pipeline would notice.

Model validation in the physical sciences. Null-model comparison is standard practice in network science, and the null-model control here (Sec. 2.2) is borrowed directly from that tradition — including the discipline of reporting *which* nuisance structure each null destroys.

What traditional sanity checks miss. The most direct way to appreciate the value of the negative-control discipline is to ask: which defects in this pipeline would *not* have been caught by conventional positivity, finiteness, or non-emptiness checks? Three concrete cases answer that question:

1. **An inverted completeness criterion** (Sec. 4.5): a guard that *passed* when the referenced control was missing and *failed* in the healthy case. Every positivity and finiteness check in the pipeline reported green throughout. Only the completeness guard — which checks that cited guard names exist in the runtime registry — could detect this, because the defect was in the *accounting structure*, not in any numerical value.
2. **A fully wired bridge carrying no upstream quantity** (Sec. 5.3): layer 4 derives $\chi_{\text{dev}} = 4$ from the area law, but layer 5 hard-codes its own bond dimension. The call chain is complete end-to-end; every function returns a finite, non-empty result. A positivity check on the curvature report would have passed silently. The gap is visible only to a control that asks whether the *upstream-derived quantity* actually flows to the consumer.
3. **A readout invariant under the fitted state** (Sec. 5.4): the MERA bulk graph is bit-identical across seeds, bond dimensions, and *states*. A sanity check asking “is the graph non-empty?” or “is curvature finite?” passes trivially. Only the explicit invariance test (rebuild and compare edge sets) reveals that the readout is an analytic property of the wiring, not an emergent geometry.

These are not hypothetical vulnerabilities. They are defects that *existed in the pipeline as built*, and that the negative-control ledger caught precisely because each control is required to be able to fail. A positivity check cannot catch what is already positive.

7 Reproducibility

Source files. The pipeline requires exactly two Python modules to run: `spiral_model_v15.py` (entry point; orchestrates all seven layers) and `spiral_metric_v15.py` (guard registry, ledger logic, metric definitions, negative-control table). No ancillary data files, precomputed logs, or external datasets are needed; every output artefact is generated from scratch.

Entry point. A single command, no arguments:

```
python spiral_model_v15.py
```

Environment and dependencies. The pipeline requires Python 3.x with `numpy`, `scipy`, and a tensor-network library (the MERA construction follows Evenbly & Vidal [2], requiring L as a power of two; this is the source of the chain-length ceiling, not an ED limit). Exact library versions are pinned in `requirements.txt` in the repository. On a single laptop the run takes on the order of 20 minutes.

Generated outputs. The run writes all artefacts to disk: `_v15_run.log` (full pipeline log, ~ 1245 s), `result/_v15_data.json` (structured metrics), `result/spiral_v15.png`, and `result/spiral_v15_metrics.png` (20-panel metric report). The guard ledger is printed to standard output. No pre-existing data is read; the pipeline is fully self-contained.

Code availability. The complete source code, `requirements.txt`, and documentation are available at: <https://github.com/yanjianzhong/spiral-emergence> (tag v15). No ancillary material is uploaded to arXiv; reproduction requires only cloning the repository and running the entry point.

Expected output. The following lines are what a correct run prints. They are the reader’s check that the reproduction succeeded:

Line	Expected
metrics meeting target	18/18
guard pass rate	54/54 = 100.00% (plus 20 diagnostics excluded, 17 of them not passing)
ledger completeness	phantom references: none; missing layers: none
negative-control layer coverage	6/7, gap ['L1']
MERA bulk geometry	Forman mean -2.242, Ollivier mean -0.3297
artefacts	result/spiral_v15.png, result/spiral_v15_metrics.png, result/_v15_data.json
wall clock	~ 1245 s; order-of-magnitude reference, machine-dependent

Table 7: Expected output for a correct reproduction.

Two cautions that belong in this section, not in a footnote. (a) These numbers certify that the pipeline *ran correctly*. They do not certify that any conclusion is *true*: 54/54 means 54 scored assertions hold. (b) The values in Table 7 are transcribed from the frozen reference run, which was re-executed *after* the most recent change to the F6d guard. The earlier inference — that renaming a diagnostic and changing its condition string cannot move the totals, because a diagnostic does not enter the denominator — was checked against that re-run and held for all three totals. The wall-clock figure did move (1138 s $\rightarrow$ 1245 s, +9.4%); it is machine-dependent and should be read as an order of magnitude only.

Data. The pipeline reads no external data for the Ising layers; the Gray–Scott exhibit uses a published dataset, cited in the source.

Scope of the accompanying repository. The code repository also contains work outside the scope of this paper, including an exploratory framework whose framing and terminology differ from the account given here. This paper is self-contained: where the two differ, the claims, numbers, and boundaries stated here govern, and nothing in the repository’s other material should be read as a result of this paper.

8 Conclusion

We stated an operational criterion for a negative control — it must be able to fail when the guarded conclusion is false — and used it as a filter rather than an aspiration: it excluded positivity, finiteness, and non-emptiness checks outright, including one candidate already in place. We sorted the survivors into four types, built a per-layer ledger, and gave the ledger a completeness guard that checks it against the *runtime* registry instead of its own text. We separated scored assertions from diagnostics so that the pass rate cannot be inflated by checks that cannot fail.

Applied to a seven-layer pipeline, the method produced a ledger covering 6 of 7 layers and, more usefully, three negative results it would have been easy to omit: a layer whose gap is

structural (nothing to push against), a bridge that is fully wired yet carries no upstream-derived quantity, and a “geometry” readout that is an analytic property of the network wiring rather than an emergent one. It also caught a real defect in which a completeness criterion was inverted — a check that passed for precisely the wrong reason, which is the failure mode this method exists to find.

The substrate physics is textbook Ising criticality, pinned by a parameter-free CFT finite-size formula at every accessible L and cross-checked by a genuinely independent spectral estimator. That is deliberate: the substrate is the *test bench*. The claim on offer is that the bench is honest — that the pipeline reports, in the same document, both what it verified and what it could not.

Falsifiability of the method itself. The ledger’s completeness guard fails if a cited guard does not exist in the runtime registry or if a layer is missing. The coverage guard fails if coverage changes. Both are checks on the accounting, which is precisely their scope: they make the ledger *hard to fake*, not the physics true. This is the method’s most distinctive property: **the validation framework is itself validated by the same standard it enforces**. It is falsifiable in exactly the way it demands of every pipeline guard. The fact that this self-check caught a real defect — an inverted completeness criterion that passed for the wrong reason — is the strongest evidence that the discipline works, because it worked *on itself*.

Reproducibility statement. Every numerical claim in this paper can be regenerated by running `python spiral_model_v15.py` from the tagged release at <https://github.com/yanjianzhong/spiral-emergence> (tag v15). No precomputed data was read; no results were cherry-picked.

A Guard accounting, verbatim

Log line, quoted without editing:

```
guards 54/54 = 100.00% (plus 20 diagnostics excluded, of which 17 did not pass)
```

Diagnostic registration is explicit per guard (`expect_pass=False`); 74 guards total = 54 scored + 20 diagnostic. Of the 20 diagnostics, 17 fail by design and 3 pass while remaining excluded from the denominator. The distinction between “failed” and “defective” is therefore carried by the data structure and not by the reader’s charity.

A methodological note, recorded deliberately. The suspicion that the bond dimension constituted an undeclared coupling between layers 4 and 5 — described in Sec. 5.3 — was raised and then falsified within seconds, because the cheapest available falsification test was named *before* any effort was spent defending the claim. This was the third time in the project that a causal claim was overturned by its own measurement. The practice of stating the cheapest falsification first is worth more than any individual result in this paper, and is recommended independently of it.

References

- [1] Brian Swingle. Entanglement renormalization and holography. *Phys. Rev. D*, 86:065007, 2012.
- [2] Glen Evenbly and Guifre Vidal. Algorithms for entanglement renormalization. *Phys. Rev. B*, 79:144108, 2009.
- [3] Pasquale Calabrese and John Cardy. Entanglement entropy and quantum field theory. *J. Stat. Mech.*, page P06002, 2004.
- [4] Glen Evenbly and Guifre Vidal. Tensor network states and geometry. *Phys. Rev. Lett.*, 107:010501, 2011.
- [5] Guifre Vidal. Entanglement renormalization. *Phys. Rev. Lett.*, 99:220405, 2007.